

# 语法分析程序的设计与实现--LL分析方法--实验报告

2023210992-周柄名

## 实验一：编程实现算法 4.2，为给定文法自动构造预测分析表

数据结构：

```
int table[100][100] = { 0 }; // 预测表
char Vt[100] = { "" }; // 终结符，额外加入“#”意为栈顶符号（“dollar符”）
char Vn[100] = { "" }; // 非终结符
string Generative[100] = { "" }; // 文法产生式存储
string GenerativeNew[100] = { "" }; // 文法产生式分解后的存储
// 文法产生式的输入（注意：文法产生式之间用空格隔开，不要换行）：A->BC C->+BC|@ B->DE E->DE|@ D->(A)|i，空用@表示
// 文法产生式的存储：GenerativeNew中的每一个字符串都是一个产生式（A->BC）

string first[100] = { "" }; // first集合
string follow[100] = { "" }; // follow集合
// first中的下标与Vn中的一致，如果A在Vn中下标为1，则A的first下标为1，存的是它的first集组成的字符串
// 可以用Back(A)获得此下标
```

- **终结符和非终结符分别用字符数组保存。**每一位保存一个终结符/非终结符预测表、first集、follow集的对应位置均相同。例如：E在Vn中下标为1的位置，那么first集中下标为1的位置保存的字符串就是E的first集组成的字符串；同理follow和table也是如此。
  - **优势：**用下标统一定位，写了专门的back函数来定位。
  - **不足：**导致符号都必须是一位字符，像E' 这种就不允许，给符号命名带来了一些麻烦
  - **其他：**输入默认第一个非终结符是起始符，保存在Vn[0]的位置。

算法流程：

1. 输入终结符、非终结符、产生式
2. 对用“|”隔开的产生式进行拆分，例如：`E->E+T|E-T` 将被拆分为 `E->E+T` 和 `E->E-T`
3. 建立first集，first集去重
4. 建立follow集，follow集去重
5. 构建预测分析表

例：

```
// 终结符号：+ - * / ( ) n 用n表示num
```

```
// 非终结符号：E T F （默认第一个是开始符号）
```

```
// 产生式：E->E+T|E-T|T T->T*F|T/F|F F->(E)|n
```

```
// 手动消除左递归后：用e表示E'，t表示T'，@表示空
```

```
// 终结符号：+ - * / ( ) n
```

```
// 非终结符号：E e T t F
```

```
// 产生式：E->Te e->+Te|-Te|@ T->Ft t->*Ft|/Ft|@ F->(E)|n
```

**不足：**未实现自动处理左递归的相关部分，需要手动处理然后输入。若存在左递归会导致first集建立函数无限循环（该函数由递归实现）。

## 运行示范：

输入：

```
请输入终结符号：
+ - * / ( ) n
请输入非终结符号：
E e T t F
请输入文法产生式：
E->Te e->+Te|-Te|@ T->Ft t->*Ft|/Ft|@ F->(E)|n
```

构建first集、follow集

非终结符的first集为：

```
FIRST(E): (n
FIRST( ):
FIRST(e): +-@
FIRST( ):
FIRST(T): (n
FIRST( ):
FIRST(t): */@
FIRST( ):
FIRST(F): (n
```

非终结符的follow集为：

```
FOLLOW(E): #)
FOLLOW( ):
FOLLOW(e): #)
FOLLOW( ):
FOLLOW(T): #)+-
FOLLOW( ):
FOLLOW(t): #)+-
FOLLOW( ):
FOLLOW(F): #)+-*/
```

预测分析  
表：

| 预测分析表为： |        |        |        |        |        |      |       |      |  |
|---------|--------|--------|--------|--------|--------|------|-------|------|--|
|         | +      | -      | *      | /      | (      | )    | n     | #    |  |
| E       |        |        |        |        | E->Te  |      | E->Te |      |  |
| e       | e->+Te | e->-Te |        |        |        | e->@ |       | e->@ |  |
| T       |        |        |        |        | T->Ft  |      | T->Ft |      |  |
| t       | t->@   | t->@   | t->*Ft | t->/Ft |        | t->@ |       | t->@ |  |
| F       |        |        |        |        | F->(E) |      | F->n  |      |  |

PS C:\Users\10901\Desktop\syntax analysis>

## 实验二：编程实现算法 4.1，构造LL (1)预测分析程序

在实验一的基础上建立分析栈，每次分析查表完成操作，如果无操作，则记录并报错，输出整个步骤。

```
stack<char> st;//预测分析栈
```

部分代码样例：当栈顶和当前符号匹配时的逻辑

```
else if (Is(st.top(), Vt, VtNum)) {
    if (st.top() == word[i]) {
        cout << "步骤" << n << "：" << "\t" << "匹配栈顶和当前符号" << word[i] << "，匹配成功；\n";
        st.pop();
        i++;
        n++;
    }
    else {
        cout << "步骤" << n << "：" << "\t" << "识别失败！！\n";
        n++;
        break;
    }
}
a = st.top();
```

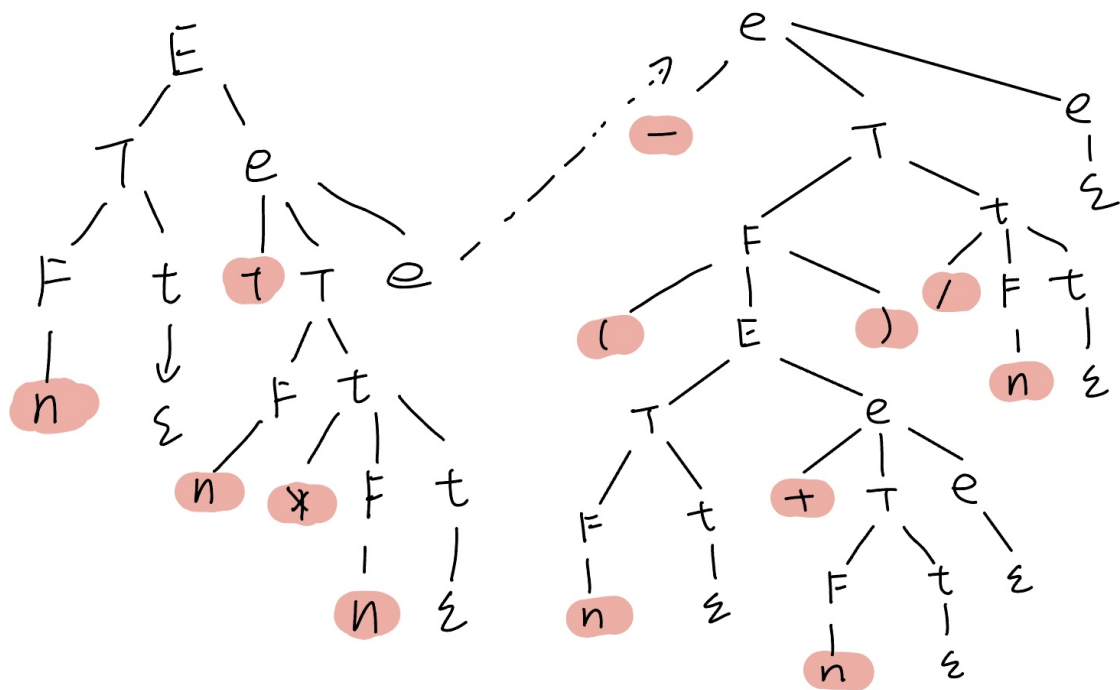
例：待分析句子为 $n+n*n-(n+n)/n$

请输入您需要测试的文字：  
 $n+n*n-(n+n)/n$   
识别结果如下：

步骤1: 使用产生式 $E \rightarrow Te$ 入栈完成；  
步骤2: 使用产生式 $T \rightarrow Ft$ 入栈完成；  
步骤3: 使用产生式 $F \rightarrow n$ 入栈完成；  
步骤4: 匹配栈顶和当前符号 $n$ ，匹配成功；  
步骤5: 使用产生式 $t \rightarrow @$ ；  
步骤6: 使用产生式 $e \rightarrow +Te$ 入栈完成；  
步骤7: 匹配栈顶和当前符号 $+$ ，匹配成功；  
步骤8: 使用产生式 $T \rightarrow Ft$ 入栈完成；  
步骤9: 使用产生式 $F \rightarrow n$ 入栈完成；  
步骤10: 匹配栈顶和当前符号 $n$ ，匹配成功；  
步骤11: 使用产生式 $t \rightarrow *Ft$ 入栈完成；  
步骤12: 匹配栈顶和当前符号 $*$ ，匹配成功；  
步骤13: 使用产生式 $F \rightarrow n$ 入栈完成；  
步骤14: 匹配栈顶和当前符号 $n$ ，匹配成功；  
步骤15: 使用产生式 $t \rightarrow @$ ；

步骤16: 使用产生式 $e \rightarrow -Te$ 入栈完成；  
步骤17: 匹配栈顶和当前符号 $-$ ，匹配成功；  
步骤18: 使用产生式 $T \rightarrow Ft$ 入栈完成；  
步骤19: 使用产生式 $F \rightarrow (E)$ 入栈完成；  
步骤20: 匹配栈顶和当前符号 $($ ，匹配成功；  
步骤21: 使用产生式 $E \rightarrow Te$ 入栈完成；  
步骤22: 使用产生式 $T \rightarrow Ft$ 入栈完成；  
步骤23: 使用产生式 $F \rightarrow n$ 入栈完成；  
步骤24: 匹配栈顶和当前符号 $n$ ，匹配成功；  
步骤24: 匹配栈顶和当前符号 $n$ ，匹配成功；  
步骤25: 使用产生式 $t \rightarrow @$ ；  
步骤26: 使用产生式 $e \rightarrow +Te$ 入栈完成；  
步骤27: 匹配栈顶和当前符号 $+$ ，匹配成功；  
步骤28: 使用产生式 $T \rightarrow Ft$ 入栈完成；  
步骤29: 使用产生式 $F \rightarrow n$ 入栈完成；  
步骤30: 匹配栈顶和当前符号 $n$ ，匹配成功；  
步骤31: 使用产生式 $t \rightarrow @$ ；  
步骤31: 使用产生式 $t \rightarrow @$ ；  
步骤32: 使用产生式 $e \rightarrow @$ ；  
步骤33: 匹配栈顶和当前符号 $)$ ，匹配成功；  
步骤34: 使用产生式 $t \rightarrow /Ft$ 入栈完成；  
步骤35: 匹配栈顶和当前符号 $/$ ，匹配成功；  
步骤36: 使用产生式 $F \rightarrow n$ 入栈完成；  
步骤37: 匹配栈顶和当前符号 $n$ ，匹配成功；

手工构造分析树验证分析结果如下：



分析正确